

Bloque 09 - SQL, NoSQL y almacenamiento

Propósito

Entender cómo viven los datos en una empresa, consultar tablas con SQL y saber cuándo un modelo documental o clave-valor exige otro diseño.

Lecciones

1. Modelo relacional, tablas y grano
2. Seleccionar, filtrar y resumir con SQL
3. JOIN y validación de cardinalidad
4. CTE, ventanas, fechas y nulos
5. MongoDB y documentos
6. DynamoDB y patrones de acceso
7. Warehouse, lakehouse y consultas asistidas

Práctica

Resuelve [la consulta de conversión](#) y compara con [la solución](#).

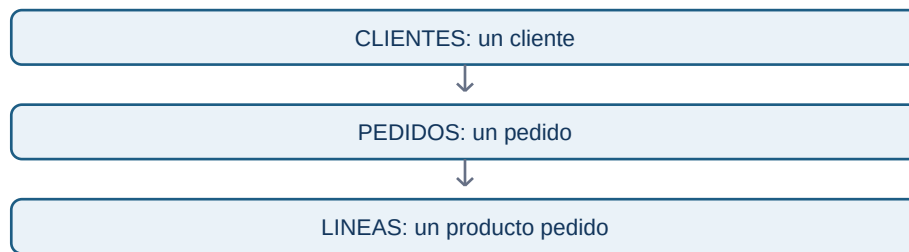
Modelo relacional, tablas y grano

Objetivos y prerequisites

Comprenderás una base de datos relacional como un conjunto de tablas conectadas, antes de escribir una consulta.

Una base de datos guarda información para que programas y personas puedan consultarla de forma consistente. En el modelo **relacional**, una tabla representa una entidad o hecho: clientes, pedidos o eventos. Una fila es una observación y una clave identifica o conecta filas.

La pregunta principal antes de SQL es el **grano**: ¿una fila de pedidos representa un pedido completo o una línea de producto? Sumar importes después de unir tablas sin responderla puede duplicar ingresos.



El diagrama muestra relaciones uno a muchos: un cliente puede tener pedidos y un pedido varias líneas. La clave no es una decoración: define qué combinaciones son válidas.

Resumen

SQL opera sobre tablas, pero el análisis depende de grano y claves. Sigue con [selección](#), [filtro](#) y [agregación](#).

Seleccionar, filtrar y resumir con SQL

Objetivos y prerequisites

Escribirás una consulta legible que responda una pregunta concreta y comprobarás qué filas excluye.

SQL es un lenguaje declarativo: describes qué resultado quieres, no los pasos internos exactos. Para contar pedidos pagados por canal:

```
SELECT canal, COUNT(*) AS pedidos
FROM pedidos
WHERE estado = 'pagado'
GROUP BY canal
ORDER BY pedidos DESC;
```

WHERE filtra filas antes de agrupar; GROUP BY define el nivel del resultado. COUNT (*) cuenta filas, pero COUNT(DISTINCT pedido_id) cuenta pedidos únicos. Elegir uno es una definición de métrica, no una preferencia sintáctica.

Error habitual

Filtrar un periodo con texto o sin zona horaria explícita puede incluir o excluir registros inesperados. Inspecciona datos de borde y declara la ventana temporal.

Resumen

Una consulta profesional hace visibles medida, población y grano. Continúa con [JOIN y cardinalidad](#).

JOIN y validación de cardinalidad

Objetivos y prerequisites

Combinarás tablas sin multiplicar accidentalmente registros.

Un JOIN une filas por una clave. INNER JOIN conserva coincidencias; LEFT JOIN conserva todas las filas izquierdas y muestra ausencia de coincidencia. Antes de unir, declara cardinalidad: uno a uno, uno a muchos o muchos a muchos.

```
SELECT p.pedido_id, c.país
FROM pedidos p
LEFT JOIN clientes c ON p.cliente_id = c.cliente_id;
```

Si clientes contiene dos filas para el mismo cliente_id, cada pedido aparecerá dos veces. Comprueba recuento antes y después, claves duplicadas y nulos introducidos por la unión. Un SQL que ejecuta no prueba que el resultado sea válido.

Resumen

La cardinalidad es un supuesto analítico que debes validar. Sigue con [SQL analítico](#).

CTE, ventanas, fechas y nulos

Objetivos y prerequisites

Escribirás consultas por pasos y compararás una fila con su contexto sin destruir el detalle.

Una CTE (WITH) da nombre a un resultado intermedio y mejora la revisión. Las funciones de ventana calculan sobre un grupo sin reducirlo: ROW_NUMBER() ordena pedidos por cliente, SUM(...) OVER (...) construye acumulados y LAG() compara con el valor anterior.

Las fechas necesitan una zona y una granularidad; los nulos no equivalen automáticamente a cero. Usa COALESCE solo cuando la regla de negocio diga qué significa la ausencia.

Resumen

La consulta mantenible separa pasos, documenta supuestos y conserva el detalle necesario para validar.

MongoDB y documentos

Objetivos y prerequisites

Entenderás qué problema resuelve un documento flexible y qué preguntas siguen siendo analíticas.

Un documento puede guardar información anidada de una entidad, por ejemplo un pedido con dirección y líneas. MongoDB almacena colecciones de documentos y permite filtros y pipelines de agregación. Esta flexibilidad ayuda cuando la forma de los datos cambia o una aplicación lee el agregado completo.

No significa “sin modelo”. Debes definir identificadores, campos opcionales, versiones y cómo se agregan importes. Documentos duplicados o esquemas inconsistentes complican análisis histórico y comparaciones.

AI asistiendo consultas

Una herramienta que genera un filtro o pipeline desde lenguaje natural produce un borrador. Revisa colección, filtros, periodos, campos sensibles, coste e interpretación del resultado antes de ejecutarlo o compartirlo.

Resumen

NoSQL cambia el modelo de almacenamiento; no elimina definición de métrica ni validación.

DynamoDB y patrones de acceso

Objetivos y prerequisites

Comprenderás por qué algunas bases clave-valor se diseñan empezando por las consultas que una aplicación necesita.

DynamoDB organiza registros alrededor de una clave de partición y, opcionalmente, una clave de ordenación. Está pensado para accesos predecibles a gran escala: “dame los pedidos de este cliente ordenados por fecha”, no para unir libremente cualquier tabla después.

Antes de modelar, enumera patrones de acceso, volumen, frecuencia y orden requerido. Diseñar solo por “entidades bonitas” puede producir consultas caras o imposibles. Para análisis amplio se suele extraer a un warehouse o lakehouse.

Resumen

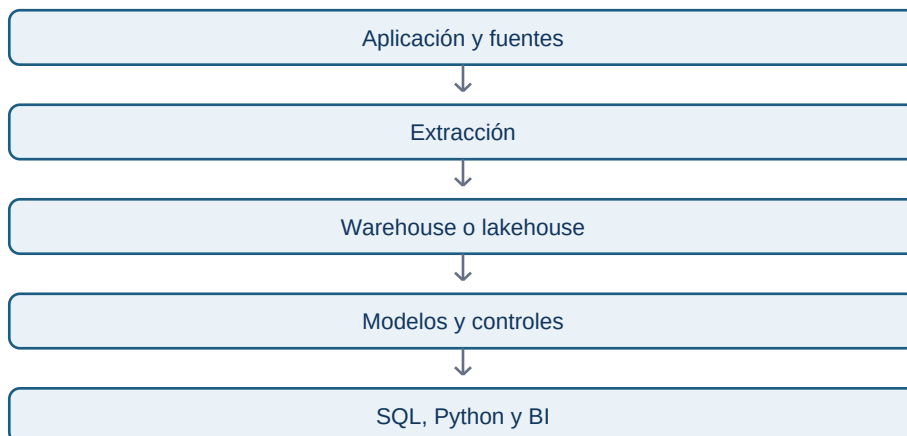
El modelo operativo optimiza accesos conocidos; el modelo analítico optimiza preguntas y historia.

Warehouse, lakehouse y consultas asistidas

Objetivos y prerequisites

Relacionarás fuentes operacionales con el entorno donde se preparan datos para análisis.

Un sistema operacional registra transacciones para que la aplicación funcione. Un **warehouse** organiza datos históricos y modelados para consulta analítica; un **lakehouse** combina almacenamiento flexible con capacidades analíticas. La arquitectura suele extraer, transformar y documentar datos antes de dashboards, SQL o Python.



La AI puede acelerar un borrador de consulta, pero no conoce por defecto el grano, la semántica, permisos o coste. Contrasta siempre resultado, plan de consulta y definición de métrica.

Resuelve la [consulta de conversión](#) antes de consultar la solución.